Обучение с учителем

Алгоритм построение модели чаще всего состоит из следующих шагов:

1 – Загрузка данных

2 – Разведывательный анализ данных

3 – Очистка данных и генерация новых признаков

4 – Разбиение выборки на обучающую и валидационную

5 – Построение модели

6 – Настройка и оценка финальной модели

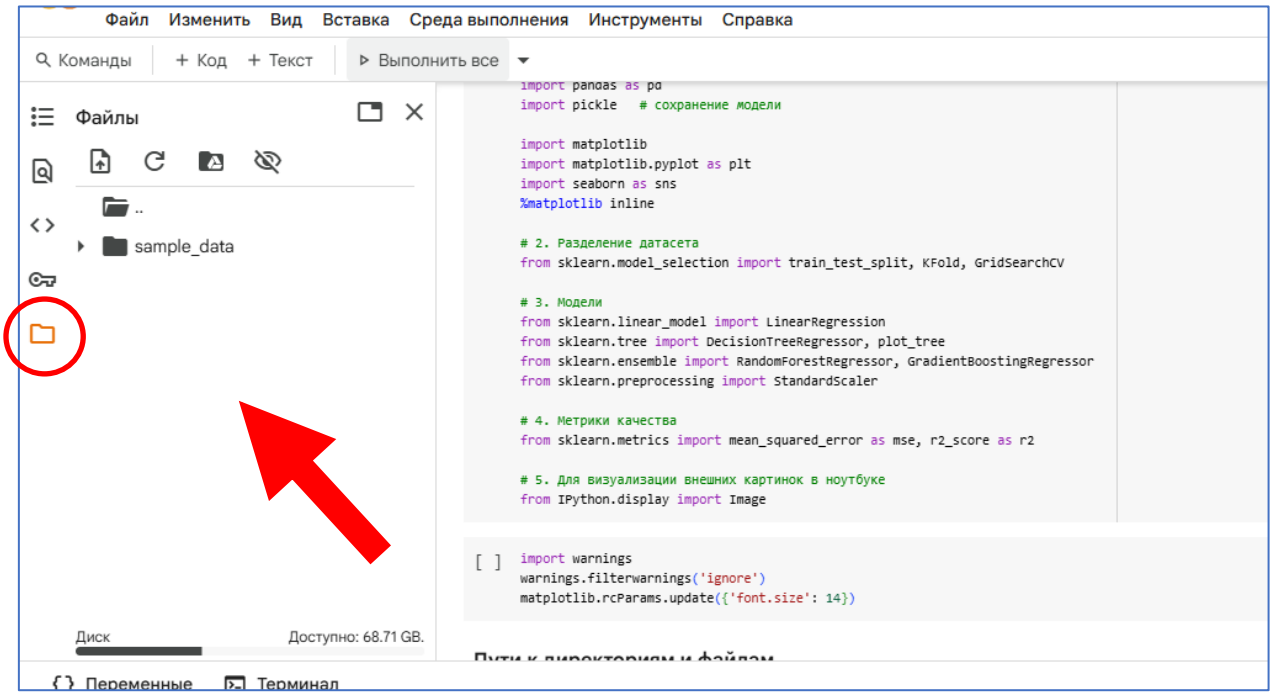
7 – Анализ прогнозов

8 – Сохранение модели

Пройдемся подробнее по каждому из шагов.

1 – Загрузка данных

Если вы пользуетесь инструментом Google Colab, то для подключения нужного файла с данными можно его перетянуть в специальную область



Еще один способ подключить файл через Google Drive.

```
[ ]  #подключить гугл диск
     from google.colab import drive
     drive.mount('/content/drive')
```

```
[ ]  #прописать вместо "..." путь к файлу train.csv
     import pandas as pd
     df = pd.read_csv('/content/drive/MyDrive/DS/Lec/train.csv')
```

2 – Разведывательный анализ данных

На этом этапе проводится знакомство с данными, знакомство с названиями признаков, их типами данных, наиболее часто используются методы библиотеки pandas .info и .describe(). Необходимо также посмотреть какие уникальные значения присутствуют в категориальных признаках, сколько их, некоторые значения могут отличаться только тем, что где-то в наборе данных категориального признака написано с заглавной буквы, но следует понимать, что для системы это будут 2 разных названия, например «minsk» и «Minsk». В таком случае для такого признака необходимо все значения преобразовать либо к заглавным буквам, либо к нижнему регистру. Подобным образом следует поизучать каждый признак.

Также желательно преобразовать типы данных, чтобы уменьшить используемую память.

3 – Очистка данных и генерация новых признаков

На этом этапе проводится проверка данных на наличие аномальных значений, пропусков (подробнее об этом написано в документе «Методы очищения данных - устранение пропусков, исправление»). Для этого используются статистические и графические методы.

Важно изучить распределение целевого признака (если решается задача обучения с учителем), в идеальном случае оно должно быть нормальным или близким к нормальному распределению.

После очистки от выбросов и заполнения пропусков (или удаления пропусков) можно переходить к генерации новых признаков.

Можно руководствоваться следующими рекомендациями. Для *непрерывных* признаков – если присутствуют признаки с высокой корреляцией их можно преобразовать при помощи нелинейных зависимостей, например, имеются признаки роста и веса, и они имеют высокую корреляционную зависимость. Тогда их можно заменить одним новым признаком индексом массы тела.

Метод Feature discretization (дискретизация признаков) в машинном обучении - это процесс преобразования непрерывных признаков в дискретные (категориальные) путем разбиения их значений на интервалы или группы (bins). Это делается для упрощения данных и повышения интерпретируемости модели. Основные аспекты метода Feature discretization:

- преобразует числовые признаки в категориальные, разбивая диапазон значений на конечное число интервалов;

- может улучшить работу моделей, особенно линейных, за счет моделирования нелинейных зависимостей через дискретные группы;

- упрощает структуру данных, что помогает уменьшить влияние шума и повысить соотношение сигнал/шум.

Допустим, есть признак "Возраст" с непрерывными значениями от 0 до 100. Для дискретизации можно разбить этот признак на 4 интервала:

0–18 лет - категория "молодой"

19–35 лет - категория "молодой взрослый"

36–60 лет - категория "взрослый"

61 и старше - категория "пожилой"

Вместо точного возраста модель будет работать с четырьмя категориями, что упрощает анализ и позволяет выявлять сегменты с разными закономерностями. Например, можно заметить, что в категории "пожилой" выше вероятность заболевания. Такой подход помогает моделям лучше справляться с задачами, где важны группы значений, а не точное число в пределах этих групп.

Метод Feature binarization в машинном обучении - это преобразование признака (обычно непрерывного или категориального с несколькими значениями) в бинарный формат с двумя возможными значениями, обычно 0 и 1. Это можно рассматривать как частный случай дискретизации, когда признаку присваивается значение 1, если он удовлетворяет определённому условию (например, превышает порог), и 0 - если условие не выполняется.

Неколичественные строковые признаки всегда нужно переводить в числовое представление, так как многие модели машинного обучения не способны работать со строками. Для кодирования категорийных признаков можно воспользоваться One-Hot-Encoding (OHE) кодированием, оно же dummy-кодирование. Для кодируемого категориального признака создаются N новых признаков, где N - число категорий. Каждый i-й новый признак - бинарный характеристический признак i-й категории. На рис. показана реализация с помощью sklearn.preprocessing.OneHotEncoder.

```
from sklearn.preprocessing import OneHotEncoder
ohe = OneHotEncoder(sparse=False)
new_ohe_features = ohe.fit_transform(data.city_le.values.reshape(-1, 1))
tmp = pd.DataFrame(new_ohe_features, columns=['city=' + str(i) for i in range(new_ohe_features.shape[1])])
data = pd.concat([data, tmp], axis=1)
data
```

	city	class	degree	income	city=0	city=1	city=2
0	Moscow	A	1	10.2	0	0	1
1	London	B	1	11.6	0	1	0
2	London	A	2	8.8	0	1	0
3	Kiev	A	2	9.0	1	0	0
4	Moscow	B	3	6.6	0	0	1
5	Moscow	B	3	10.0	0	0	1
6	Kiev	A	1	9.0	1	0	0
7	Moscow	A	1	7.2	0	0	1

Когда не хотят заполонять признаковую матрицу большим количеством бинарных признаков, применяют кодировки, в которых категории кодируются какими-то интерпретируемыми значениями (Feature encoding -Описание категории каким-то числом). Например, если это категория товаров в интернет-магазине, то логично её закодировать средней ценой товара. Тогда, по крайней мере, наш новый признак упорядочивает категории по дороговизне.

```
# функция возвращает значения нового признака
def code_mean(data, cat_feature, real_feature):
    return (data[cat_feature].map(data.groupby(cat_feature)[real_feature].mean()))

data['city_mean_income'] = code_mean(data, 'city', 'income')
data
```

	city	class	degree	income	city_mean_income
0	Moscow	A	1	10.2	8.5
1	London	B	1	11.6	10.2
2	London	A	2	8.8	10.2
3	Kiev	A	2	9.0	9.0
4	Moscow	B	3	6.6	8.5
5	Moscow	B	3	10.0	8.5
6	Kiev	A	1	9.0	9.0
7	Moscow	A	1	7.2	8.5

Самый простой способ кодирования - заменить каждую категорию числом входящих в неё объектов (т.е. знания других признаков вообще не нужно). Следует понимать, это не самый лучший вариант кодирования.

После того, как набор данных подготовлен к моделированию его целесообразно сохранить.

4 – Разбиение выборки на обучающую и валидационную

Метод train_test_split в машинном обучении – это процесс разделения исходного набора данных на две части: тренировочную (train) и тестовую (test). Тренировочная часть используется для обучения модели, а тестовая для её оценки на новых, ранее не виденных данных, чтобы проверить качество и способность модели обобщать знания.

Основные моменты train_test_split:

- обычно данные разделяются в пропорции 70-80% на обучение и 20-30% на тест, это помогает избежать переобучения модели и оценить её реальную эффективность;

- в Python функция train_test_split из библиотеки scikit-learn реализует этот процесс автоматически;

- разделение может проводиться случайным образом или с сохранением пропорций классов (стратифицированный разбиение).

Помимо train и test, может также выделяться validation set – для настройки гиперпараметров модели.

Синтаксис функции train_test_split следующий:

```
train_test_split(*arrays, test_size=None, train_size=None, random_state=None, shuffle=True, stratify=None)
```

где

arrays – последовательность массивов или структур данных одинаковой длины для разбиения;

test_size – доля или количество данных, выделяемых под тестовую выборку;

train_size – доля или количество данных, выделяемых под обучающую выборку;

random_state – значение для инициализации генератора случайных чисел (для воспроизводимости);

shuffle – параметр, определяющий нужно ли перемешивать данные перед разбиением;

stratify – массив меток классов для стратифицированного разбиения и сохранения распределения классов.

Пример:

```
X = df[feature_names]# массив признаков
y = df[target_name] #целевая переменная
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33,
shuffle=True, random_state=42)
```

5 - Построение модели

Ниже будут приведены примеры и проведено сравнение качества разных моделей. Для того, чтобы можно было сравнить качество моделей необходимо ввести такое понятие, как метрика качества. Для решений задач регрессии обычно используются такие метрики, как MSE, MAE и R^2.

Метрика MSE (Mean Squared Error, среднеквадратичная ошибка) – это показатель качества регрессионной модели, который измеряет среднее значение квадратов разностей между предсказанными и фактическими значениями.

Метрика MSE рассчитывается по формуле

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

где

y_i - реальное значение целевой переменной,

$\hat{y}_i$- предсказанное моделью значение,

n – количество наблюдений;

Особенности:

– усиливает влияние крупных ошибок, так как ошибка возводится в квадрат, что делает метрику чувствительной к выбросам;

– всегда неотрицательна, значение 0 соответствует идеальному прогнозу без ошибок;

– используется для оценки точности модели и оптимизации её параметров во многих алгоритмах машинного обучения;

– имеет единственный глобальный минимум, что облегчает использование для градиентного спуска и других оптимизационных методов.

Метрика MAE (Mean Absolute Error, средняя абсолютная ошибка) – это показатель качества регрессионной модели, который измеряет среднее значение абсолютных разностей между предсказанными и фактическими значениями.

Описание метрики MAE:

рассчитывается как среднее арифметическое абсолютных значений ошибок:

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

y_i - реальное значение целевой переменной,

$\hat{y}_i$ – предсказанное моделью значение,

n – число наблюдений;

Особенности:

– отражает средний размер ошибки без учёта её знака, то есть перемещения вверх или вниз;

– менее чувствительна к выбросам по сравнению с MSE, так как ошибки не возводятся в квадрат;

– имеет ту же размерность, что и исходные данные, что облегчает интерпретацию;

– используется для оценки точности модели и сравнения различных моделей.

Метрика R-квадрат (коэффициент детерминации) – это статистическая мера, показывающая долю дисперсии зависимой переменной, объясненную моделью с помощью независимых переменных.

Формула R-квадрат:

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

Описание метрики R-квадрат:

– принимает значения от 0 до 1, где 0 означает, что модель не объясняет вариации данных, а 1 – что модель объясняет всю вариацию;

– показывает, насколько хорошо построенная модель описывает наблюдаемые данные;

– используется для оценки качества регрессионных моделей;

– высокое значение R-квадрат свидетельствует о хорошей подгонке модели, но не гарантирует причинно-следственную связь.

Таким образом, R-квадрат измеряет долю вариативности зависимой переменной, которую объясняет модель, и помогает понять, насколько хорошо модель отражает данные.

линейной модели после разбиения данных на обучающую и валидационную выборки включает несколько шагов:

разбиение данных – исходный набор данных делится на две части: обучающую (train) и валидационную (validation) с помощью функции train_test_split.

обучение модели – на обучающей выборке создаётся и обучается линейная регрессионная модель. В процессе обучения модель находит оптимальные коэффициенты, которые минимизируют ошибку предсказания.

оценка модели – с помощью валидационной выборки проверяется, как хорошо модель предсказывает ответ для данных, которые не использовались при обучении. Это позволяет оценить обобщающую способность модели и избежать переобучения.

Пример построения линейной модели:

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# X - признаки, y - целевая переменная
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2,
random_state=42)

# создание и обучение модели
model = LinearRegression()
model.fit(X_train, y_train)

# предсказания на валидационной выборке
y_pred = model.predict(X_val)

# оценка качества модели
mse = mean_squared_error(y_val, y_pred)
print(f"mean squared error on validation set: {mse}")
```

Синтаксис методов fit и predict, которые используются для обучения и предсказания моделей в Scikit-learn, следующий:

Метод fit(X, y)принимает на вход X – матрицу признаков (обучающие данные) и y – целевую переменную (метки или значения). Обучает модель, вычисляя внутренние параметры (например, коэффициенты в линейной регрессии) и затем возвращает объект модели с обученными параметрами.

Метод predict(X) принимает новую выборку X (например, валидационные или тестовые данные). На основе обученной модели возвращает предсказанные значения целевой переменной. Этот методы используется после вызова fit для предсказаний. Таким образом, fit готовит модель к работе, а predict применяет модель для предсказаний.

Более подробное описание этих методов можно почитать, перейдя по ссылке https://scikit-learn.ru/getting_started/

В этом примере для оценки модели используется метрика MSE. Но для оценки качества модели одного значения этой метрики недостаточно. Необходимо иметь значения метрики как для тестового набора значений, так и для обучающего. Если разница метрик небольшая это говорит о том, что модель обучилась хорошо и не переобучилась. Так, например, если имеются такие значения метрик после обучения линейной модели на обучающем наборе данных:

```
R2:   0.596
RMSE: 61716.907
MSE:  3808976573.954
```

и на валидационном:

```
R2:     0.596
RMSE:   62833.745
MSE:    3948079567.649
```

то выводы будут следующими: т.к. R2 = 0.596, это говорит о том, что только 59,6% целевой переменной объясняется включенными в модель факторами, т.е. модель слабо описывает зависимость целевого признака от факторных переменных. Другими словами, модель находится в зоне недообучения. Считается, что модель неплохого качества, если R2>0,7. Также мы видим, что разница метрик на обучении и на валидации очень незначительна, что является хорошим признаком. Если разница метрик составляет более 10%, то это является свидетельством того, что модель переобучилась, т.е. нашла зависимости, которых нет.

Примером переобучения могут служить следующие значения метрик на обучении:

```
R2:   1.0
RMSE: 0.0
MSE:  0.0
```

на валидации:

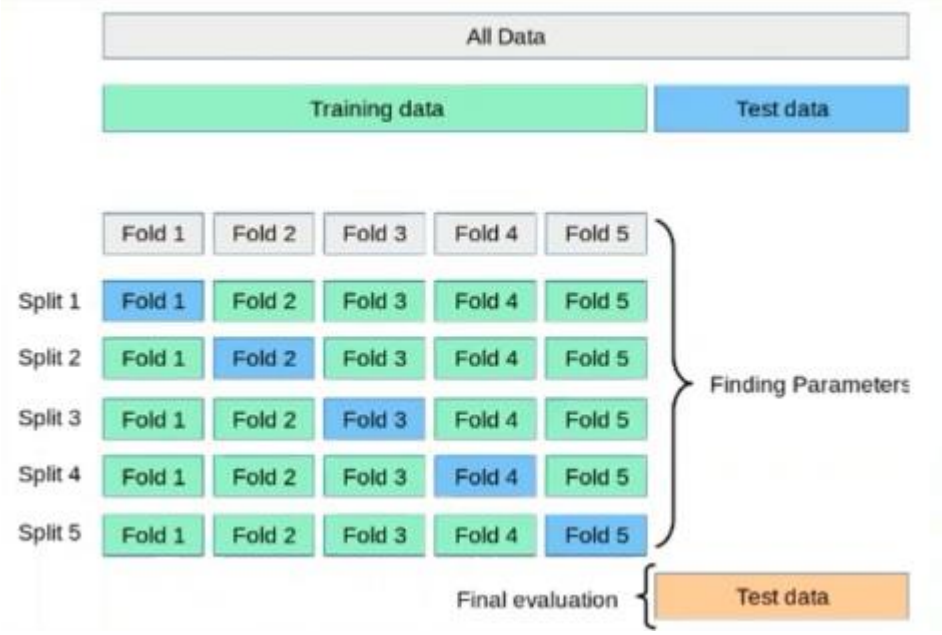
```
R2:   0.782
RMSE: 45362.918
MSE:  2057794356.979
```

То, что на обучении метрика R2 = 1 и MSE = 0, уже является поводом заподозрить переобучение. Так же мы видим разница метрик составляет более 20%, что является свидетельством того, что модель явно переобучилась, хотя и валидационная метрика достаточно высокая, что говорит о неплохом качестве модели.

6 – Настройка и оценка финальной модели

Кросс-валидация – это способ проверить, насколько модель машинного обучения хорошо будет работать на новых данных. Для этого исходный набор данных разбивают на несколько примерно равных частей. Модель обучается на большинстве этих частей, а одну часть временно оставляют для проверки. Затем процесс повторяют, каждый раз используя другую часть для проверки, а остальные – для обучения. Такой подход позволяет получить среднюю оценку качества модели, учитывая все данные, и избегает переобучения.

Кроссвалидация проходит на обучающих данных.



Суть кросс-валидации:

– исходные данные разбиваются на несколько частей (фолдов);

– модель обучается на части данных и тестируется на оставшейся;

– процесс повторяется для всех частей, каждая часть один раз используется для теста;

– оценки качества модели усредняются по всем фолдам.

Пример, создаем модель градиентного бустинга. В примере законмментированны 2 гиперпараметра, которые будем подбирать в процессе кроссвалидации.

```
gb_model = GradientBoostingRegressor(criterion='squared_error',
                            min_samples_leaf=10,
                            random_state=42,
                           # n_estimators=100,
                           # max_depth=7
                           )
```

Используем GridSearchCV для поиска наилучших значений гипермараметров. В примере создается словарь с двумя ключами (гиперпараметры со значениями, которые будут перебираться). Получается,

что у нас будет построено 16 моделей: модель градиентного бустинга из 50 деревьев глубиной 3, 50 деревьев глубиной 5, 50 деревьев глубиной 7 и 50 деревьев глубиной 10; аналогично для 100 деревьев, 200 и 400. В качестве метрики выбран коэффициент детерминации, набор данных будет разбит на 5 фолдов и последний гиперпараметр n_jobs=-1 указывает, что будут нагружены все ядра процессора.

```
params = {'n_estimators':[50, 100, 200, 400],
          'max_depth':[3, 5, 7, 10]}

gs = GridSearchCV(gb_model, params,
                  scoring='r2', # метрика
                  cv=KFold(n_splits=5,    # k (кол-во разбиений/итераций) в
кросс-валидации
                           random_state=21,
                           shuffle=True),
                  n_jobs=-1
                  )
gs.fit(X_train, y_train)
```

Данная процедура достаточно продолжительна по времени, т.к. требует построения большого количества моделей и оценки их качества. После расчетов вы модем получить список лучших параметров относительно выбранной метрики

```
gs.best_params_
```

```
{'max_depth': 7, 'n_estimators': 200}
```

Данную модель остается только обучить с указанием лучших значений гиперпараметров

```
final_model = GradientBoostingRegressor(criterion='squared_error',
                                        max_depth=7,
                                        min_samples_leaf=10,
                                        random_state=42,
                                        n_estimators=200)

final_model.fit(X_train, y_train)
```

7 – Анализ прогнозов

На данном этапе мы можем оценить важность признаков

```
final_model.feature_importances_
```

```
array([0.10791977, 0.04809955, 0.05191513, 0.01432036, 0.02552001,
       0.03751747, 0.02311576, 0.45662056, 0.00259454, 0.17193605,
       0.00110564, 0.00819186, 0.0212251 , 0.0299182 ])
```

Обернув этот массив в DataFrame можно отсортировав его получить 3 самых важных признака и 3 самых неважных.

8 – Сохранение модели

Сохраняем обученную модель final_model в файл с именем 'model' в бинарном режиме. Это делается с помощью модуля pickle, который сериализует объект модели, преобразует его в поток байтов и записывает в файл. Такой файл можно затем загрузить и использовать без повторного обучения модели.

```
with open('model', 'wb') as file:
    pickle.dump(final_model, file)
```

Пример загрузки модели из файла:

```
with open('model', 'rb') as file:
    loaded_model = pickle.load(file)
```

Это позволит восстановить модель в рабочее состояние и делать предсказания на новых данных. Использование pickle – стандартный способ сохранения и загрузки моделей в Python для последующего использования.